

MIRAGE

by Ahmed Alqalam, Andrew Cassarino, Nicolo Krueger
Biomedical Engineering, George Washington University Student
Expected graduation May 2025

A Senior Capstone Project Overview submitted to

The Faculty of
The School of Engineering and Applied Science
of the George Washington University
in partial fulfillment of the requirements
for the degree of Bachelor of Science

May, 2025

© Copyright April 2022
All rights reserved

Abstract

The primary goal of this project is to address the critical challenges faced by radiologists, including burnout caused by increased workloads, work hours, and demand for faster report turnaround times, especially when the case is complex. It was estimated that between 54% to 72% of diagnostic and interventional radiologists experience burnout symptoms. Several factors contribute to this overwhelming discovery found within radiology. Higher expectations for report turnaround time, understaffing, and longer workdays are all main contributing factors to the spike in burnout rates we see. Suppose diagnostic radiologists had a resource that could quickly give a recommended diagnosis and then write a report autonomously. This would save crucial time for radiologists, allowing them to spend more time interacting with patients, other physicians, NP's, and PAs, increasing the overall productivity of the hospital.

Our solution is powered by an AI-mediated engine we created: MIRAGE (Medical Retrieval-Augmented Generation Environment). MIRAGE is a medical retrieval augmented generation (RAG) system that will expedite interpreting X-ray results, alleviating the cognitive burden on radiologists. This will allow users to focus their efforts and spend more time collaborating with healthcare providers to improve productivity and patient care outcomes in hospitals. The product is designed to integrate seamlessly within the hospital's workflows, which minimizes disruption and maximizes efficiency. This solution was inspired by insights from our client, Dr. Syed Anwar, at the Children's National Hospital, who emphasized the need to reduce manual labor in identifying complex and rare cases, which addresses one of the root causes of radiologist burnout.

<https://github.com/drewreno/BMEGWU2025>

Table of Contents

1.1 Contents

Table of Contents	iv
1.1 Patient Population.....	1
1.2 Overview of Project Goals.....	1
1.3 Objectives, Functions, and Specifications.	1
1.4 Existing Products.....	2
1.5 Solution.....	2
1.6 Definitions of Success.....	2
2.1 Technical Overview	3
2.1.1 CORPUS	3
2.1.2 Retrievers	4
2.1.3 Input	4
2.1.4 Large Language Model (LLM) and Output	5
2.2 Technical Challenges.....	6
2.3 System Testing.....	7
3.1 Test Results	9
3.2 Accomplishments.....	10
3.3 Setbacks.....	10
3.4 Discussion.....	11
3.5 Recommendations & Future Work	11
4 User Instructions/Manual.....	11
5. Appendices	15
6. Reference	17

List of Figures

Figure 1 - Mirage System Architecture.....	3
Figure 2 - Template Snippet.....	5
Figure 3 - Input/Output Debug 1.....	8
Figure 4 - Input/Output Debug 2.....	8
Figure 5 - Round 1 Test Results.....	9
Figure 6 - Round 2 Testing Results.....	10

List of Tables

Table 1 - Mapping Radiology Related Requirements to Corresponding User Stories	1
Table 2 - Computational Power Solutions	6

CHAPTER 1 – BACKGROUND

1.1 Patient Population

Radiologists, particularly those working in high-demand hospital settings, are increasingly experiencing severe burnout due to growing workloads, long hours, and demand for rapid report turnaround, especially when the case is complex. It was estimated that between 54% to 72% of diagnostic and interventional radiologists experience burnout symptoms (Fawzy). This issue affects not only the mental health and career longevity of radiologists, but also the efficiency of hospital systems and ultimately patient outcomes. The U.S. healthcare system faces an economic burden as radiologist diagnostic delays increase. According to a 2018 Harvard report, physician burnout results in an estimated \$990,000 in lost revenue per departed physician, with replacement costs ranging between \$500,000 and \$1,000,000 (Interlad). This is a financial strain that impacts hospital operations and healthcare efficiency. Our project is specifically designed to support this population by reducing the time and cognitive effort required for interpreting chest X-ray radiographs.

1.2 Overview of Project Goals

To directly address the radiologist burnout crisis, our goal is to design an AI-powered platform called MIRAGE (Medical Retrieval-Augmented Generation Environment). The projects aim to streamline radiology workflows by assisting in interpreting chest X-rays using a retrieval augmented generation (RAG) system. Users will benefit from increased diagnostic efficiency, reduced manual workload, and faster-decision making. This project is inspired by real-world needs identified through collaboration with our client, Dr. Syed Anwar at Children’s National Hospital.

User Stories:

- As a radiologist, I need AI-generated insights to prioritize critical cases.
- As a radiologist, I need a system that reduces the time spent on writing reports while maintaining accuracy.
- As a physician, I want to ensure patient information is confidential.
- As a physician, I want accessible and accurate pre-interpreted X-rays to expedite treatment planning, especially in cases of complex diagnoses or when radiologists face high workloads and burnout.

1.3 Objectives, Functions, and Specifications.

Our key objectives are to: deliver accurate, context-aware diagnostic support; enable multimodal inputs (texts and image); Ensure ease of use via a friendly user interface; and protect patient confidentiality through local deployment.

Table 1 - Mapping Radiology Related Requirements to Corresponding User Stories

Requirement	User Story	Solution
Rapid insight generation	Radiologist prioritizing critical cases.	<1 minute response time.
Ease of use	Reduce manual reporting	UI accommodates text/image input
Data Security	Maintain confidentiality	Fully local deployment
Medical Accuracy	Support diagnosis	Corpus and prompt structure tuned for radiology.

1.4 Existing Products

Existing products and technologies in the domain of medical imaging AI offer diverse capabilities, tailored to specific applications and use cases. BiomedParse, developed by Microsoft Research and the University of Washington, is an AI model that analyzes medical images to predict systemic diseases, allowing users to interact with the system using plain English queries. MedImageInsight, an embedding model, excels in classification and similarity search across various medical imaging modalities such as radiology, pathology, and ophthalmology, streamlining workflows. RadCLIP leverages Contrastive Language-Image Pre-training (CLIP) and incorporates a unique 3D slice pooling mechanism, enhancing volumetric image analysis in radiology. The open-source framework MONAI supports deep learning applications in healthcare by providing robust tools for image processing, model development, and evaluation. Finally, InnerEye, a Microsoft Health Futures initiative, focuses on automatic and quantitative 3D image analysis, aiming to democratize AI for medical imaging stakeholders.

In contrast, Med Rag differentiates itself from traditional AI models, which typically focus on classification, segmentation, or diagnostic predictions based solely on pre-trained knowledge. Traditional models often require retraining to incorporate new medical insights and operate within a closed system. Med Rag integrates external knowledge retrieval, such as clinical notes or medical guidelines, and augments generative capabilities by continuously linking to an up-to-date corpus, minimizing retraining needs. This approach offers cost efficiency and rapid access to relevant knowledge. However, its effectiveness is dependent on the quality and breadth of the external knowledge base, and it raises considerations around data privacy.

1.5 Solution

MIRAGE is a radiology-centered retrieval-augmented generation system that accepts multimodal inputs, chest X-rays and text, to generate context-specific, high-confidence diagnostic suggestions. Our biggest challenge has and will continue to be the model's accuracy. The entire reason for choosing a RAG centric methodology is to prioritize accuracy over time to generate an output. Doctors are unlikely to require the model to output something in under a minute, especially if it is at the cost of accuracy. As such, our solution framework to combat receiving proper diagnosis from a radiologist every time an x-ray is taken has been molded to maximize accuracy. Another issue with generative AI is it is incapable of stating it is unsure about what it just said. Instead, it will always dispense an output with 100% certainty, which is rarely good for medical applications. Our options are limited but one of which is to maximize accuracy to where the certainty problem merely becomes an outlier. Another solution would be to alter the prompting to make it choose multiple possible diagnoses and give a percentage confidence. Only issue being that without any concrete answer to judge whether its statements are true, iterating for the sake of accuracy would become a far more complex process.

1.6 Definitions of Success

Our project is considered successful if it meets several high-level goals. These include reducing report turnaround time, minimizing cognitive load on radiologists, and increasing diagnostic accuracy and confidence – which typically early-stage clinical natural language processing systems (NLPs) achieve 70 % to 75 % accuracy. In addition, this system must be intuitive and accessible to clinicians who may not have technical expertise.

CHAPTER 2: TECHNICAL JUSTIFICATION

2.1 Technical Overview

The function block diagram is a graphical language for programmable logic controller design, that can describe the function between input variables and output variables. As shown in Figure 1, the user interface (UI) developed serves as an entry point for users to input their inquiries. These can be asking the system questions for the identification of specific pathologies or asking the system questions about treatment plans such as “What imaging characteristics suggest the need for surgery in this scenario?” This triggers the retrieval process that uses complex algorithms, such as MedCPT, to extract relevant information from the corpus. The corpus, provided by Dr. Anwar, includes anonymized MIMIC data comprising patient reports. The information retrieved would then be processed by Large Language Models (LLMs) such as GPT or LLama to generate specific-context evidence-based responses. The processed answer would then be delivered to the user via the UI providing insights based on the question asked. This workflow is designed to meet the needs of radiologists with an interface that's intuitive and allows efficiency in the execution of tasks. The user-friendly UI ensures all the complex operations are hidden, allowing the users to focus only on the insights delivered by the system.

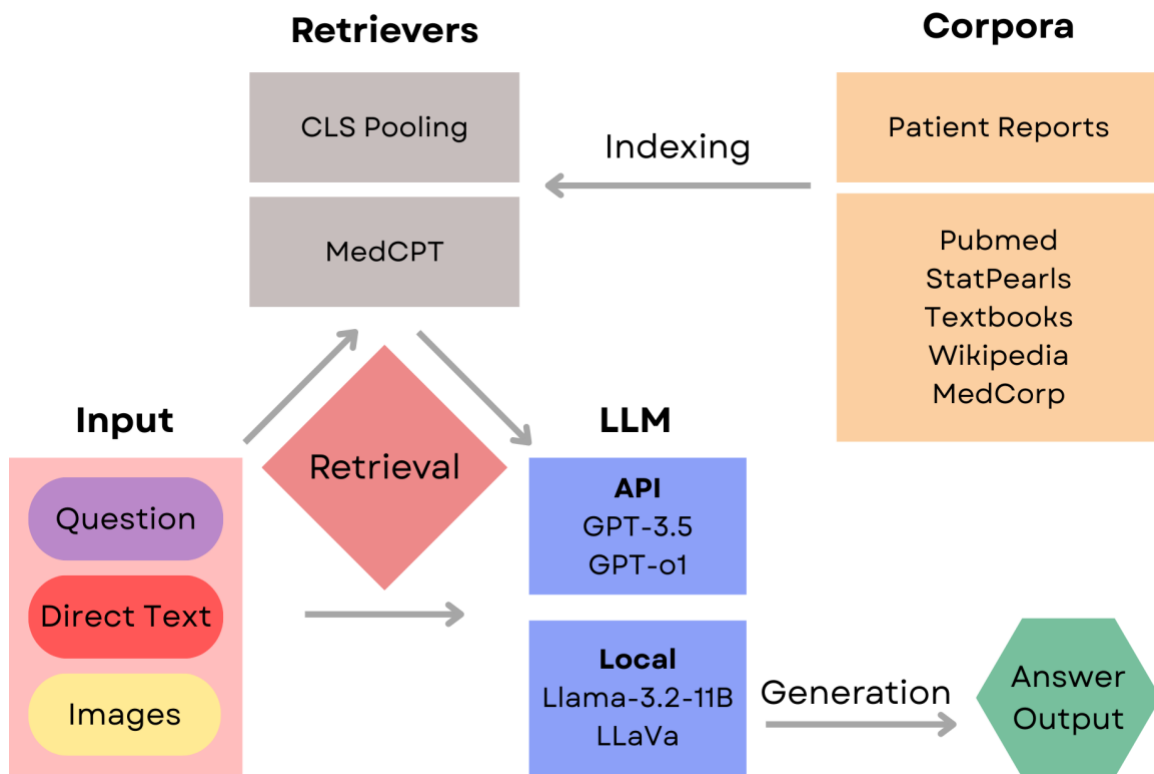


Figure 1 - Mirage System Architecture

2.1.1 CORPUS

One of the subsystems within our model is that of the corpus. The corpus is a set of files

used to prompt the model with the required knowledge for specialized uses, such as medical diagnosis. There are many ways to format said set of data, but the primary goal is to maximize the number of viable and verified data to the LLM.

To understand why we prioritize quantity over quality, we must first look at how an LLM functions. Because it is simply a large set of multiplications, limiting the number of inputs can actually be to our own detriment. Many LLM transformer matrices are multiple GB of information long so when we only input a sentence or two of our own information, it is much more difficult to control and influence the output of the model. Another noteworthy feature of what makes a corpus important is that it allows for nonspecialized models to be used for specialized tasks. A few reasons why this is so beneficial is that it is incredibly expensive to train your own LLM and requires a team of professionals to be able to verify its effectiveness and continue to steer it towards greater success. The most notable of which though is the ever-improving models, meaning self-trained ones would also need constant iteration to keep up. Instead, by prioritizing proper data and input management, we can continue to use the latest models for our software.

Another key detail we had to navigate when designing our input system was how the LLM creates its output. Simply put, it tries to predict the next most likely set of words given those from the input and its own matrix. This means that the information most recently given to it is weighted more heavily as such, we place the corpus at the beginning of our input. This allows the specific input to our query to be prioritized over the generalized blanket of information provided in the corpus.

2.1.2 Retrievers

Another subsystem within our model is the retrievers. Retrievers are tools that are responsible for querying the corpus based on semantic that is similar and relevant to the user's input. They return a list of passages or documents that are fed into the generative model. MIRAGE specifically utilizes CLS Pooling and MedCPT which are transformer-based retrieval models that are optimized for medical domains. Said passages are separated into chunks for proper processing, allowing the model to not misinterpret the corpus as key data to an individual. The retriever is also responsible for the indexing of the corpus which can be used in case a specific ordering or formatting of the corpus is requested. Given we had more time, one of the future goals with the project was to create a corpus for a variety of domains and pick the corresponding one for our specific usage. This system would heavily rely on the retrieval process to achieve such a system, but since we never set this as one of our key goals and merely something the customer would like to adapt our product to do, the retrieval process stayed simple.

One of the challenges we faced with pooling was with how we formatted our corpus. MedCPT allows for articles to be ripped from their source from libraries such as Pubmed to be used in a corpus. This would be a great system to utilize for the future development of MIRAGE, however, for our specific uses, we instead simply wanted localized, hand-picked, data. As such we pivoted to using CLS which rips the text from the local corpus directly.

2.1.3 Input

This subsystem includes the question submission module where users can input questions, direct clinical reports, or upload radiological images. The input was by far the most

important section to get right because it contributes most to whether the MIRAGE system gives reliable outputs or not.

We employed many techniques to ensure the input was properly understood by the model.

```
# Define the templates and options
general_medrag_system = '''
    DO NOT DISCUSS FILES WITHIN **CORPUS**

    You are a helpful medical expert, and your task is to evaluate a medical report and/or image.
    Please first think step-by-step describing relevant findings in the critical image(s) and report(s).
    Organize your output with json(section[str, str]) and list the number of images received (may be 0).
    Your responses will be used for research purposes only, so please have a definite answer.
    ...

# Start of corpus is more heavily weighted
general_medrag = Template('''

Here are the background knowledge documents, do not reference this data in your response:
<CORPUS>
{{ context }}
</CORPUS>

Here is the question and critical information used in your response. Overweigh the text:
{{ question }}

Here are the potential choices. Give each a percentile confidence:
{{ options }}

Please think step-by-step and generate your diagnosis in json (must be 100+ tokens if images are present):
''')
```

Figure 2 - Template Snippet

Above in Figure 2 is one of the sections within our template to help package our input along with the corpus properly to the LLM. You can see that the corpus comes first, then the {{question}} section which both includes the images and text pertaining to the specific input we are doing. And finally, it ends with the most critical part of the input which tells the model exactly what we want it to give us back. In this case, a json output with at least 100 tokens of response with a diagnosis of the patient.

One of the more complex parts of the system is how the images are sent and understood by the model. Image pattern recognition in AI has been around for a while, but since we are working with generalized LLMs and include a large and critical text input it was a significant challenge to ensure that our system still functioned as we require. Images are incredibly large from a data perspective meaning there was a serious concern images could overpower our corpus and direct instructions to the model. However, with proper formatting we discovered that the LLMs available have already seemed to solve this problem which is why it's a deliverable resource now in engines such as chat-GPT. Even so, different models "see" images differently making it one of the few sections of our code which had to be rewritten to properly interact with the localized models.

2.1.4 Large Language Model (LLM) and Output

The final subsystem, the LLM, is by far the most important since it is the AI behind our system. It accepts both the query (input) and the retrieved text (corpus), feeding them into

a large language model, such as GPT or LLama. When discussing which model we should use for our project it was apparent that there was not one end-all answer. LLaMA-3.2-11B offers a high level LLM rivaling that of modern GPT models, however it requires an immense amount of your own computational power. After some research and quick testing, we found that an even simpler (less intensive) model of LLama, Llama-3, ran at 1 token per second on our personal computers. Given our system operates in the 5-to-6-digit level of tokens, we decided to start with GPT instead.

GPT models work differently since they are privatized models and can only be accessed through a server. This works to our benefit since we instead upload our input and corpus to their own computers making responses go from hours to seconds. However, since we want our system to not upload any confidential information, our end goal still requires the localized model LLama provides. As a compromise, we instead used the API service of GPT to test our model since the data source we were using was anonymized specifically for testing purposes.

Once we were happy with the model's accuracy and it came time to look at hospital implementation, we were able to adapt the systems we had been working on to a local model just as we had hoped. This came with its own set of challenges, most of which revolved around the lengthy run-time turning simple debugging into hour long endeavors. However, local models run similarly from our research and attempt to implement another model, LLava. This means that with the localized version of our software functional, only small changes will need to be made to incorporate the newest LLM.

2.2 Technical Challenges

When coming into this project early research seemed to prove that computational power would be our greatest challenge. Because corpus systems work better the larger the input, we knew we had to maximize this as much as was feasible. This on its own was not an unreasonable challenge, as with API systems we could just throw more money at our problems and solve them. However, we needed MIRAGE to work in a hospital where no confidential information can be uploaded outside their local servers. Unfortunately for us, there is not a way to follow these restrictions without completely localizing the system itself.

To address our computational power dilemma there are a few options.

1. Reduce the input size
2. Use a smaller LLM
3. Wait longer for an output
4. Get more computational power

Each of these solutions comes with its own benefits and downsides, making a mix of the three the best solution to the problem.

Table 2 - Computational Power Solutions

Solution	Benefits	Downsides
Reduce Input	• Powerful LLM	• Limits corpus, reducing accuracy • Constrains image size
Smaller LLM	• Keeps input intact • Takes less disk storage	• Reduces accuracy

Wait	• Most accurate and uncompromising version of software	• Doctors need responses in a reasonable time
More VRAM	• Optimal solution	• Costs money

In a best-case scenario, the hospital could simply ask for a larger budget for our system and expand its computational power so no compromises must be made. However, limiting input size does make logical sense as the larger the corpus the less of an impact each token will have on the output. Essentially, it's a problem of diminishing returns so we chose an arbitrary limit of 50,000 tokens which from a brief period of testing seemed to net the best blend of what we were looking for. Even with the limit set, minimizing the runtime is vital so the other options were also weighed.

Using a smaller LLM seems like a possible solution, however there is a certainty that the model will be less accurate if we employ this. Because the generalized models undergo thorough testing, there is not a company on the market who releases a larger and less competent model. Since accuracy is by far our most important characteristic—more so than time—we decided not to compromise on the power of the model and used the best available.

Unfortunately for us, this means that running the system will take a long time on computers not specifically designed for running LLMs. Going into this project we knew that the hospital had adequate technology for a powerful LLM based system, so for our specific client we did not have to concern ourselves too much, however there is a significant financial commitment to employing our system to facilities that may not already have the computational power. Talking to our client, it sounded like the system we were using was upwards of 10,000 USD, so by no means a small purchase. Fortunately, if this is outside the budget of another possible client, we still have the other options mentioned earlier and would most likely just make them wait longer for a response.

2.3 System Testing

When coding it is inevitable that you encounter errors. Whether it be something as simple as syntax or even the resources themselves being uploaded or processed incorrectly. We knew this going into the project, so adding debug errors were vital to the success of our project.

Below in Figures 3 and 4 are some examples of the input and output that we see in the terminal. There are many unnecessary details to the user that are not carried over to the UI, but for testing and finding errors were vital. Here is the direct link to an example of one input/output which comprises of a two-image, one text file upload.

<https://raw.githubusercontent.com/drewreno/BMEGWU2025/refs/heads/API/resources/TERMINAL%20IO.txt>

CHAPTER 3:

3.1 Test Results

We tested our system 2 times on a MIMIC Data Set. This data set contained around 10000 chest x-rays of patients who had an abnormal chest Xray's indicating the patient was sick and healthy patients with regular chest X-Rays. Each image in the database was tracked in an excel sheet. Each row in the excel file has the following associated values study_id, subject_id and ref_id. The study_id represents the number associated to a specific image. The subject_id just represents a random number associated with the folder that holds patient folders. The ref_id is what actually represents a specific patient. In addition to these each row had a column the had a question type, question and answer. Each patient had a certain number of questions that were associated with it. A patient could have multiple images and multiple questions for each image, which is why the study_id number is so valuable. When testing we made sure to only test our system on questions that had yes or no answers. The reason for this is so that when we do analysis, we can distinctively say that our system was correct/incorrect. If we were to have done a word matching analysis, then we would've had to interpret if our system's answer was correct or not which could cause confusion. The first time we tested our system the model we used was chat-GPT 3.5 model. The total number of questions we tested was 839. Below are the results for the first round of testing.

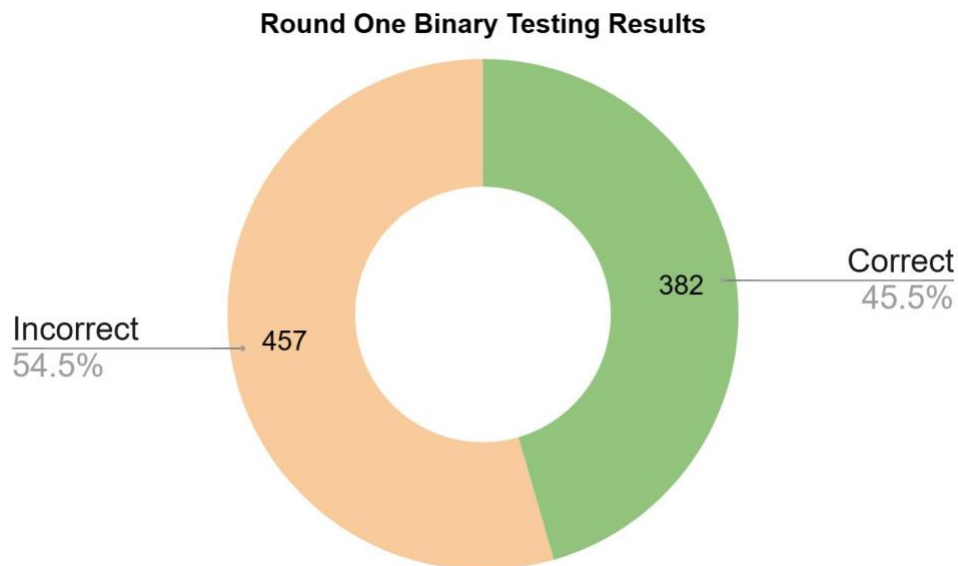


Figure 5 - Round 1 Test Results

The second time we tested our system the model we used was chat-GPT 01 model. The total number of questions we tested in the second round was 95. We tested less questions due to time constraints.

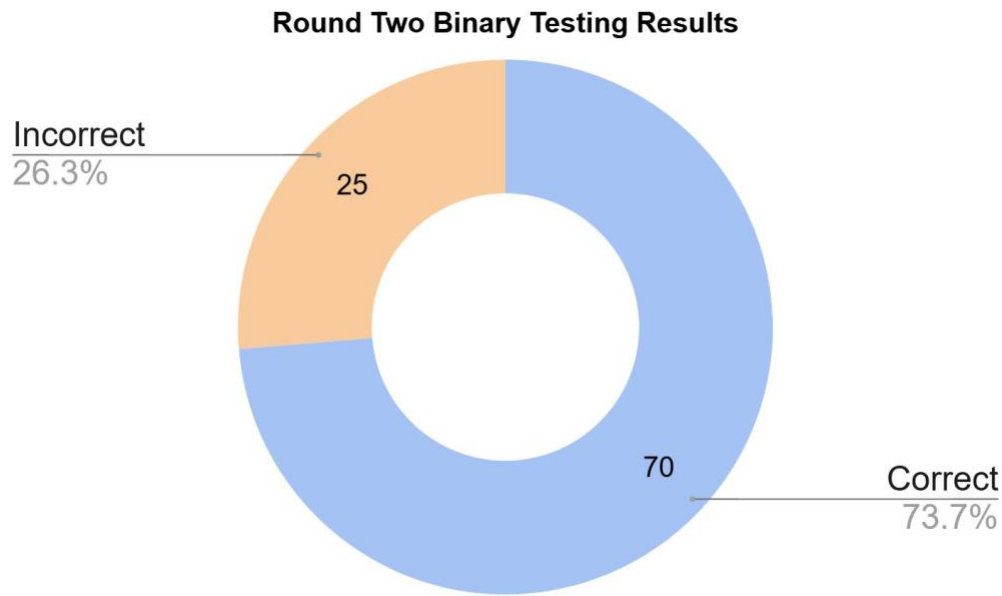


Figure 6 - Round 2 Testing Results

3.2 Accomplishments

We were successfully able to accomplish the stated goals in section 1.2 of the report. While we were not able to get feedback from a radiologist, we can evaluate if we met the user stories that were stated above. One user story was gaining AI-generated insights from our system. At the state of our current system radiologists can get suggestions and insights from our system. Additionally, radiologists wanted to ensure their patient information is confidential, which under our improved system will be, when run on a local environment. Unfortunately, we were not able to test if our system could write a report. We were able to show our clients the system and they were very happy with the system. Additionally, a correctness rate of 73.8% is something we are extremely happy about. Comparing these results to lots of the professional models, we have a quite comparable correctness rate. We able to run our system on their local environment and give us decisive answers with confidence intervals. The one issue that we had while running on the local system was anonymizing the patient information in its response. There were times when our system would give us answers and include patients names in the aanswer,which is a main concern for us.

3.3 Setbacks

There were 3 areas where our group fell short. The first was the amount of data analysis that we were able to perform. Obviously, data analysis couldn't be done until the system was completed, but we could've prepared better for when the system was finished by knowing what type of tests we wanted to do and also finding a way to automate our system. Planning ahead and allotting more time for analysis could have helped us make more progress. The second area where we fell short was treatment suggestion. A goal we set at the beginning of this project was creating a system that could not only diagnosed patients but would also suggest treatment options depending on that diagnosis and patient information. We were not able to program this into our system in time and

think this would be a great spot for the group next year to try and achieve. The last aspect that we fell short of was being able to locally integrate the Llava model into our system. Llava is a local model similar to Llama but it is a medical model, meaning it was trained on medical data. This is another great spot for the group next year to take off from and try and do next.

3.4 Discussion

We felt as if we were quite prepared. The 2 pieces of advice we would've given to ourselves was 1. Making sure to get a consistent meeting time with our client weekly. We found it challenging to meet with the client on several occasions due to scheduling conflicts and 2. Allotting more time for analysis like we mentioned above. We all feel like we are leaving this project in a great state and achieved almost all of our stated goals, we touched on this in the previous section.

3.5 Recommendations & Future Work

We mentioned in section 3.3 where we think would be a good place for the next group to pick up and the next steps to take. From a design perspective something that would really propel the project and system forward would be making it more accessible to LLM's. Trying to integrate different LLM's into the system was quite difficult and caused lots of hiccups when trying to run our system, additionally better models are released we want to be able to quickly integrate them and use them. From a testing perspective once binary testing has been done and the results are satisfactory, we thought it would've been very effective to also run some sort of word matching test. Rather than direct word matching it would be something like key word matching so words that are key to a diagnosis must be present otherwise we know the diagnosis is incorrect.

CHAPTER 4: User Instructions/Manual

4.1 Introduction

This manual provides a step-by-step guide for clinicians, researchers, or hospital IT staff on how to install, configure, and operate the MIRAGE system. This system is designed to streamline chest X-ray implementation by integrating multimodal inputs with AI-generated diagnostic suggestions. No programming experiences are required to use the MIRAGE interface.

4.2 System Requirements

- Disk Space: >50GB
- GPU RAM: >20GB recommended
 - We tested on 40GB which took around 60 seconds
- Python 3.10-3.12
- Docker (for container-based deployment)

4.3 Installation Instructions

The source code is hosted at: <https://github.com/drewreno/BMEGWU2025>

There are two branches:

- API version: uses OpenAI's GPT-o1 via API
- Local version: uses local hosted models (LLaMA-3.2-11B and LLaVA-Med)

Option A: API version (Recommended for Lightweight Use)

1. Clone the repository and switch to the API branch

In terminal, input:

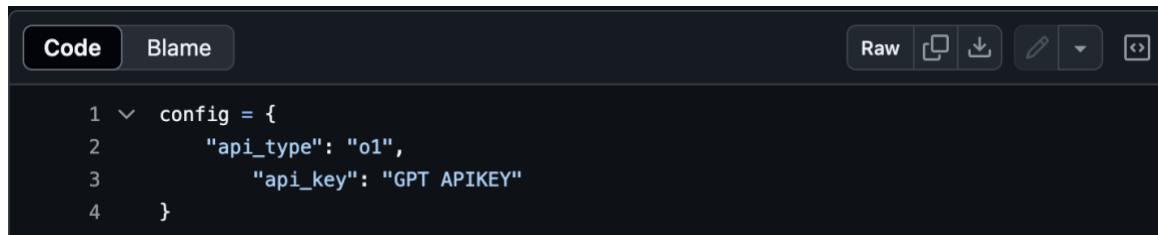
```
git clone https://github.com/drewreno/BMEGWU2025.git && cd BMEGWU2025
```

Then, again in terminal, input:

```
git checkout API
```

2. Set up the environment variables

- OpenAI API key required.
- Make a new one at: <https://platform.openai.com/api-keys>
- Open config.py
- Replace “GPT APIKEY” with “your API key”.



```
1  config = {  
2      "api_type": "o1",  
3      "api_key": "GPT APIKEY"  
4  }
```

3. Launch the interface

In terminal, input:

```
python Main_UI.py (for the old, simple UI)
```

```
python Final_UI.py --gradio_ui (for the updated, server-based UI)
```

Option B: Local version (For Offline/Private Use)

1. Clone the repository and switch to the API branch

In your terminal input:

```
git clone https://github.com/drewreno/BMEGWU2025.git && cd BMEGWU2025
```

Then, again in terminal, input:

```
git checkout Llama
```

2. Ensure model weights are downloaded in the models/ directory.

3. Create a virtual environment and install dependencies:

In terminal, input:

```
python3 -m venv venv && source venv/bin/activate
```

Then, again in terminal, input:

```
pip install -r requirements.txt
```

4. Launch the interface

In terminal, input:
python Main_UI.py

4.4 User Interface Overview

Once the system is running, open your browser to <http://localhost:9801>.

The screenshot displays a web-based user interface with a dark theme. It is divided into two main columns: 'Inputs' on the left and 'Output' on the right.

Inputs Section:

- Radiology Images:** A file named '02aa804e-bde0afdd... .jpg' (1.4 MB) is uploaded. Below it is a preview of a chest X-ray image.
- Report Files:** A file named 's50414267.txt' (644.0 B) is uploaded.
- Additional Notes:** A text area containing the word 'test'.
- Continue Button:** A large green button at the bottom of the inputs section.

Output Section:

- Textbox:** A large text area displaying a detailed medical report. The report includes sections for 'Cardiomediastinal silhouette', 'Examination of the bony thorax', 'The remainder of the visualized ribs', 'The visible parts of the upper abdomen', 'There are bilateral nodular-appearing densities', 'No suspicious parenchymal nodules or masses', 'Overall, no focal airspace process', 'The hilar regions are unremarkable', 'No signs of interstitial edema or vascular congestion', 'No acute cardiopulmonary process is evident', 'The likely explanation for the subtle bilateral nodular opacities is nipple shadowing rather than a true parenchymal lesion', 'There is no evidence of infection or an acute rib injury', 'The chronic rib deformities of the left sixth and seventh ribs appear stable', and a 'confidence_assessment' section.

Sections of the UI:

- **Query Input:** Type a clinical question that is able to retrieve information from past patient reports in the corpus.
- **Image Upload:** Drag and drop chest X-ray images (JPG format)
- **Report Files Upload:** Drag and drop patient report (.txt format)
- **Continue Button:** Executes retrieval and generation.
- **Output Textbox:** View AI-generated diagnostic suggestion and which reports from the past it retrieved to generate the response.

4.5 How to Use the System

1. Launch the interface using the setup method
2. Enter your query
3. Upload a chest X-ray image (optional but recommended for image-based reasoning)
4. Upload patient report (optional)
5. Click continue
6. Review the output

4.7 Updating the CORPUS

The system uses a curated corpus of .txt files organized in the corpus/textbooks/en directory. To update the corpus:

1. Navigate to the relevant corpus directory in the branch you're using. For example:

```
cd corpus/textbooks/end
```

2. Add your new.txt files. Each file should contain clean, structured medical content.
3. Ensure file name is consistent and description.
4. Re-index the corpus using the provided script under the directory of scr/data/:

```
Python textbooks.py
```

5. Restart the system to apply changes.

4.8 Troubleshooting

Issue	Solution
UI not loading	Ensure port 7860 is free. Restart Streamlit app
Image not accepted	Ensure format is JPG and file size is under ...
Patient report not accepted	Ensure format is txt file.
Output irrelevant	Rephrase the question or verify CORPUS is up-to-date

5. APPENDICES

5.1. Objectives, Functions, and Specifications.

Our goal was a 70% accuracy which could run in under 5 minutes on the hospital's localized LLM system. We achieved an accuracy greater than this goal on our second iteration of the software and as time goes on and LLMs improve we expect that this will only improve. The only change we made to our objective was to limit the scope to chest radiographs since we had an extensive database specific to this image/report type.

5.2. Standards.

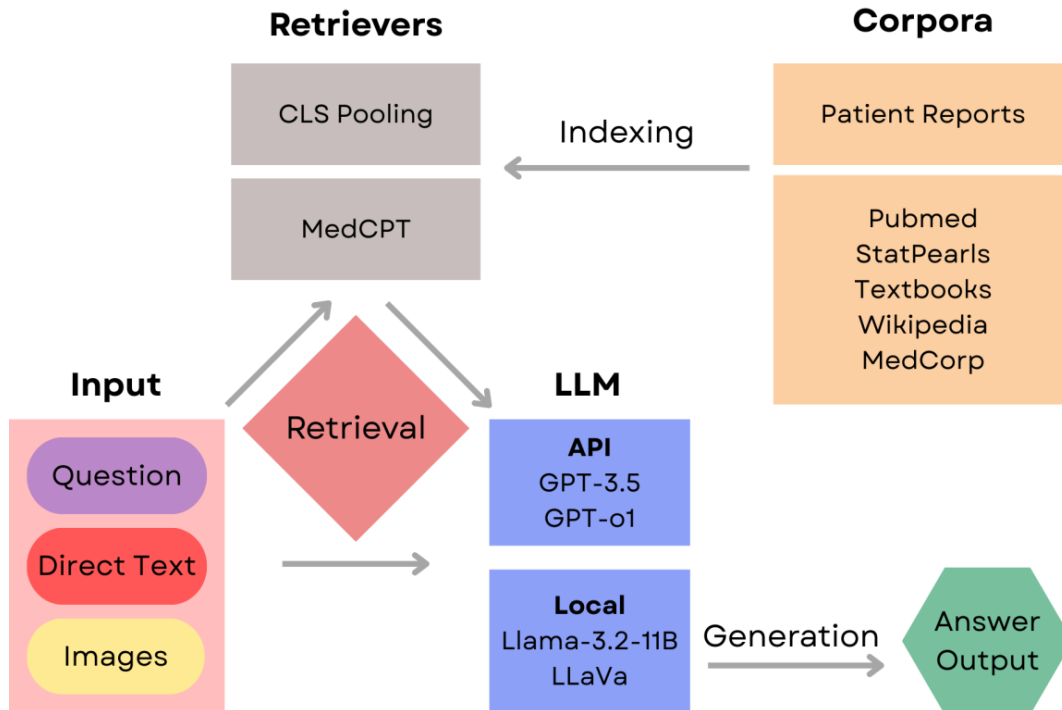
We didn't use any consensus standards to test or design our device. If our device was actually to go into the field it would have to abide by the following standards.

- HIPAA
- AAMI
- FDA REGULATIONS
- Data Privacy
 - NIST

5.3. Code.

<https://github.com/drewreno/BMEGWU2025>

5.4. Engineering Drawings.



5.5. Test Data. Present the raw data that you analyzed in Section 4. The organization of the test data should parallel the organization of your requirements and specifications in Subsection 4.1.

```
# Define the templates and options
general_medrag_system = '''

DO NOT DISCUSS FILES WITHIN **CORPUS**

You are a helpful medical expert, and your task is to evaluate a medical report and/or image.
Please first think step-by-step describing relevant findings in the critical image(s) and report(s).
Organize your output with json(section[str, str]) and list the number of images received (may be 0).
Your responses will be used for research purposes only, so please have a definite answer.
'''

# Start of corpus is more heavily weighted
general_medrag = Template('''

Here are the background knowledge documents, do not reference this data in your response:

<CORPUS>
{{ context }}
</CORPUS>

Here is the question and critical information used in your response. Overweigh the text:
{{ question }}

Here are the potential choices. Give each a percentile confidence:
{{ options }}

Please think step-by-step and generate your diagnosis in json (must be 100+ tokens if images are present):
''')
```

```
> & "C:/Users/Andrew.Cassarino/AppData/Local/Programs/Python/Python310/python.exe" "C:/Users/Andrew.Cassarino/Downloads/BPEGM2025-main/BPEGM2025-main/Pe-BAG-
main/src/Main_UI.py"
2025-04-21 18:30:57,136 - DEBUG - OpenAI API key has been set.
2025-04-21 18:30:57,137 - WARNING - process_selected_files() signature doesn't accept prompt-calling without it.
2025-04-21 18:30:57,142 - DEBUG - Image C:/Users/Andrew.Cassarino/OneDrive/Desktop/classes/CODE/capcode/corpus/images/02aa80Me-bdeafdd-112c0b34-7bc16630-4c384014.jpg encoded successfully.
2025-04-21 18:30:57,145 - DEBUG - Image C:/Users/Andrew.Cassarino/OneDrive/Desktop/classes/CODE/capcode/corpus/images/1744110c-4ec4c1f7-36a260b7-c5f994f8-79ef1962.jpg encoded successfully.
2025-04-21 18:30:57,145 - INFO - Encoded 2 images.
2025-04-21 18:30:57,145 - DEBUG - Read content from C:/Users/Andrew.Cassarino/OneDrive/Desktop/classes/CODE/capcode/corpus/question/50814267.txt.
2025-04-21 18:30:57,145 - DEBUG - Initializing MedAG instance with LLM: ol
2025-04-21 18:30:57,452 - INFO - Load pretrained SentenceTransformer: nchi/medKPT-Query-Encoder
No sentence-transformers model found with name nchi/medKPT-Query-Encoder. Creating a new one with CLS pooling.
2025-04-21 18:30:57,857 - DEBUG - Entered the answer method.
2025-04-21 18:30:57,857 - DEBUG - Question:
EXAMINATION: CHEST (PA AND LAT)
FINAL REPORT
INDICATION: ___ f with new onset ascites // eval for infection
TECHNIQUE: Chest PA and lateral.
COMPARISON: None.
FINDINGS:
There is no focal consolidation, pleural effusion or pneumothorax. Bilateral
nodular opacities that most likely represent nipple shadows. The
cardiomediastinal silhouette is normal. Clips project over the left lung,
potentially within the breast. The imaged upper abdomen is unremarkable.
Chronic deformity of the posterior left sixth and seventh ribs are noted.
IMPRESSION:
No acute cardiopulmonary process.
Batches: 1000
2025-04-21 18:32:09,591 - DEBUG - Retrieved snippets: [{"id": "InternalMed Harrison 16179", "title": "InternalMed Harrison", "content": "Radiographic Findings Although the initial chest radiograph may be normal when patients have mild symptoms, the classic radiographic appearance of PCP consists of diffuse bilateral interstitial infiltrates that are perihilar and symmetric (Fig. 244-20); yet another finding that is not specific for PCP. The interstitial infiltrates can progress to alveolar filling (Fig. 244-20). High-resolution chest CT shows diffuse ground-glass opacities in virtually all patients with PCP (Fig. 244-20). A normal chest CT essentially rules out the diagnosis of PCP. Cysts and pneumothoraces are common chest radiographic findings (Fig. 244-20). A wide variety of atypical radiographic findings have been described, including asymmetric patterns, upper lobe infiltrates, mediastinal adenopathy, nodules, cavities, and effusions."}, {"id": "InternalMed Harrison 19941", "title": "InternalMed Harrison", "content": "One way to help distinguish between crackles associated with alveolar fluid and those associated with interstitial fibrosis is to assess for egophony. Egophony is the auscultation of the sound \"AP\" instead of \"EEE\" when a patient phonates \"EEE.\" This change in note is due to abnormal sound transmission through consolidated parenchyma and is present in pneumonia but not in IPF. Similarly, areas of alveolar filling have increased whispered pectoriloquy as well as transmission of larger-airway sounds (i.e., bronchial breath sounds in a lung zone where vesicular breath sounds are expected). The lack or diminution of breath sounds can also help determine the etiology of respiratory disease. Patients with emphysema often have a quiet chest with diffusely decreased breath sounds. A pneumothorax or pleural effusion may present with an area of absent breath sounds.\""}, {"id": "InternalMed Harrison 2756", "title": "InternalMed Harrison", "content": "asthma, or pericardial disease should be considered. During the general examination, signs of anemia (pale conjunctivae), cyanosis, and clubbing (spider"}]
```

```

k=', 'detail': 'high'}}]]]
2025-04-21 18:32:49,601 - DEBUG - Received response: ChatCompletion(id='chatcmpl-B0tyUPId5yZegyDpSjV1bWG50BMc0', choices=[Choice(finish_reason='stop',
index=0, logprobs=None, message=ChatCompletionMessage(content='{
  "numberOfImagesReceived": "2",
  "step_by_step_analysis": "1) The provided PA and lateral chest radiographs show clear lung fields with no focal parenchymal consolidation, no pleural effusion, and no pneumothorax. 2) The bilateral nodular opacities at the lower lung zones are likely nipple shadows rather than true parenchymal lesions. 3) The cardiomeastinal silhouette is normal in size and contour. 4) There is a suggestion of surgical clips in the left breast region, consistent with postsurgical changes or biopsy. 5) The upper abdomen portion of the images appears unremarkable. 6) There is noted chronic deformity of the posterior aspects of the left sixth and seventh ribs, which is likely an old or healed process rather than an acute finding. 7) Overall, no evidence of acute cardiopulmonary abnormality is identified.",
  "findings_impression": "No acute cardiopulmonary process is detected on these chest radiographs.",
  "diagnosis_choices_confidence": {
    "Null": "100%"
  }
}'), refusal=None, role='assistant', annotations=[], audio=None, function_call=None, tool_calls=None)], created=1745274738, model='o1-2024-12-17', object='chat.completion', service_tier='default', system_fingerprint='fp_2ef8962a57', usage=CompletionUsage(completion_tokens=947, prompt_tokens=50704, total_tokens=51651, completion_tokens_details=CompletionTokensDetails(accepted_prediction_tokens=0, audio_tokens=0, reasoning_tokens=704, rejected_prediction_tokens=0), prompt_tokens_details=PromptTokensDetails(audio_tokens=0, cached_tokens=0)))
2025-04-21 18:32:49,601 - DEBUG - Exiting the answer method with answers: [{'numberOfImagesReceived': "2", "step_by_step_analysis": "1) The provided PA and lateral chest radiographs show clear lung fields with no focal parenchymal consolidation, no pleural effusion, and no pneumothorax. 2) The bilateral nodular opacities at the lower lung zones are likely nipple shadows rather than true parenchymal lesions. 3) The cardiomeastinal silhouette is normal in size and contour. 4) There is a suggestion of surgical clips in the left breast region, consistent with postsurgical changes or biopsy. 5) The upper abdomen portion of the images appears unremarkable. 6) There is noted chronic deformity of the posterior aspects of the left sixth and seventh ribs, which is likely an old or healed process rather than an acute finding. 7) Overall, no evidence of acute cardiopulmonary abnormality is identified.", "findings_impression": "No acute cardiopulmonary process is detected on these chest radiographs.", "diagnosis_choices_confidence": { "Null": "100%" } }']
2025-04-21 18:32:49,602 - INFO - Answer generated successfully.

```

5.6. Technical Assembly Instructions

We discuss how to set up the system in chapter 4, however if someone wanted to run the software we made, it requires some expertise in python since there are likely to be unavoidable errors when first implementing the code. When implementing the software at the hospital we were fortunate to be working with a computer scientist (Nishad K.) who was able to ensure that the environment was set up the same as ours with only some input here and there from us.

When designing the system, however, we tried to prioritize ease-of-use since not only did we have to run it across all three of our personal computers, but the hospitals as well. Although it could be made easier and there is some room for improvement, we are happy with the deliverable and think anyone with sufficient python, GitHub, and hugging-face knowledge could get it running.

References

- Fawzy NA, Tahir MJ, Saeed A, Ghosheh MJ, Alsheikh T, Ahmed A, Lee KY, Yousaf Z.
- Hugging Face, *Sentence-Transformers*, Hugging Face, 2024. [Online]. Available: <https://huggingface.co/sentence-transformers>
- Incidence and factors associated with burnout in radiologists: A systematic review. *Eur J Radiol Open*. 2023 Oct 23;11:100530. doi: 10.1016/j.ejro.2023.100530. PMID: 37920681; PMCID: PMC10618688.
- Intelrad. (2023). Burnout in Radiology: Understanding and Addressing the Challenge. Retrieved from <https://www.intelrad.com/en/articles/burnout-in-radiology/>
- Meta, *Llama 3.2 11B Vision Instruct*, Hugging Face, 2024. [Online]. Available: <https://huggingface.co/meta-llama/Llama-3.2-11B-Vision-Instruct>
- Microsoft, *LLaVA-Med: Large Language-and-Vision Assistant for Biomedicine*, GitHub repository, 2024. [Online]. Available: <https://github.com/microsoft/LLaVA-Med>
- National Center for Biotechnology Information, *MedCPT: Medical Contrastive Pretraining Transformer*, GitHub repository, 2023. [Online]. Available:

<https://github.com/ncbi/MedCPT>

- OpenAI, *OpenAI Platform Documentation: Concepts*, 2024. [Online]. Available: <https://platform.openai.com/docs/concepts>
- T. Xiong, *MedRAG: Medical Retrieval-Augmented Generation*, GitHub repository, 2024. [Online]. Available: <https://github.com/Teddy-XiongGZ/MedRAG>